



中山大學
SUN YAT-SEN UNIVERSITY

C++多态

中山大学计算机学院



主讲人：郑贵锋

zhenggf@mail.sysu.edu.cn



中山大学MOOC课程组

目录

CONTENTS

01

虚函数及其意义

02

多态的概念

03

多态-函数重载

04

多态-动态类型

05

抽象类

06

虚析构

07

RTTI运行时类型识别机制

08

final 关键字 (C++11)

09

多态的实现原理



问题引入

场景故事：门外有一些动物带着面具，你知道它们都是吃货。你想知道它们各自喜欢吃什么，于是你逐一问它喜欢吃什么。

我们把这个场景写成程序 (`what_do_you_prefer.cpp`)

明明 `cat` 和 `dog` 知道自己喜欢什么，输出答案却是令人失望

因为提问者认为它是动物，就把它们降级为吃货。这不符合常理！

显然，用 `cat` 和 `dog` 对象自己的回答替换动物通用的回答才是合理的结果。



虚函数 (Virtual Function) 概念 (1)

虚函数是一个类的**成员函数**，前面有关键字 **virtual**

- 作用：在**公有继承层次**中的一个或多个**派生类**中对虚函数进行**重定义**。当**派生类的对象**使用它**基类的指针（或引用）**调用虚函数时，将调用该对象的**成员函数**。

案例：程序 (what_do_you_prefer.cpp) 中，在 Animal 类的 eat 成员函数添加关键字 virtual，再编译、运行！

cat 说喜欢鱼，dog 说喜欢骨头。Perfect!



虚函数 (Virtual Function) 概念 (2)

“一旦为虚，永远为虚”

- 含义1：在公有继承层次中，某个类的成员函数声明为 virtual。则其**直接**或**间接**派生类中的**相同签名**的函数，都是虚函数。**不论**是否用关键字 virtual 再次申明。
- 含义2：任何虚函数，都有可能被其派生类再次重新定义，重新定义后依然是虚函数。因此，虚函数既可以被继承，也可以被重新定义。



虚函数 (Virtual Function) 意义

新的故事：来了一个新动物 bear 分类为 Bear，但不知道它喜欢什么

- 练习1（虚函数继承）：添加一个新类 Bear 公有继承 Animal。创建一个对象 bear，也问他喜欢吃什么。
- 练习2（虚函数重写）：经调查，发现bear喜欢蜂蜜（Honey）。给Bear添加方法 eat()，输出“ I like honey!”

开放扩展 (*Open for extension*) 。设计师根据基类写的函数（模块），不仅适用于已知的Cat,Dog，而且可扩展到Bear, Elephant，甚至未知的动物



覆盖 (Override) 与隐藏 (Overwrite) (1)

覆盖 (override) - **修改**基类函数定义

- 基类或非直接基类，至少有一个成员函数被 virtual 修饰
- 派生类虚函数必须与基类虚函数有同样签名，即函数名，参数类型，顺序和数量都必须相同。

隐藏 (overwrite) - **屏蔽**基类的函数定义

- 派生类的函数与基类的函数同名，但是参数列表有所差异。
- 派生类的函数与基类的函数同名，参数列表也相同，但是基类函数没有 virtual 关键字。



覆盖 (Override) 与隐藏 (Overwrite) (2)

对象指针 (引用) 被向上转型到基类指针 (引用)

- 覆盖: 调用该对象的虚函数
- 隐藏: 调用基类的函数

派生类对象赋值到基类对象

- 按基类行为调用该函数
- 练习 (赋值给基类): 将 whatDoYouPrefer 参数该为传值

覆盖 (Override) 与隐藏 (Overwrite) (3)

	作用域	是否虚函数	函数名	函数签名
重载 (Overload)	相同	-	相同	不同
隐藏/屏蔽 (Overwrite)	不同	- 否	相同	不同 相同
覆盖 (Override)	不同	是	相同	相同

目录

CONTENTS

01

虚函数及其意义

02

多态的概念

03

多态-函数重载

04

多态-动态类型

05

抽象类

06

虚析构

07

RTTI运行时类型识别机制

08

final 关键字 (C++11)

09

多态的实现原理



编程语言中多态 (polymorphism) 概念 (1)

程序语言中，**多态**特指一个**标识符**指代**不同类型**的数据。或者说，由具体的数据类型决定该标识符的语义。

例如：一桌**兄弟**吃饭，你发出**消息**“吃饭”（OOP中调用对象成员函数也称发消息）

- 兄弟1: 执行“中国人.吃饭”程序，去拿筷子
- 兄弟2: 执行“美国人.吃饭”程序，去拿刀叉
- 兄弟3: 执行“印度人.吃饭”程序，伸出了手
- 兄弟4: ...

这里 Brother& 类型对象的吃饭行为，由它指代的实际对象的吃饭程序的定义决定。如果对象，如兄弟1，的类型是 中国Brother，自然做出拿筷子行为。



编程语言中多态 (polymorphism) 概念 (2)

程序语言中，哪些**标识符**有哪些会满足多态定义？

函数重载 (Function Overload)

- *函数名*，它可为不同类型函数 (注：函数类型声明为 $f(t_1, t_2 \dots)$)

方法覆盖 (Method Override)

- 有虚函数基类的 *指针或引用*，它可指代派生类的对象

泛型 (Generics) / 模板 (Template)，即“参数化类型”

- *模板名*，例如 `vector` 可泛化指代各种类型数据的数组

编程语言中多态（polymorphism）概念（3）

多态标识符必须指派具体的函数或方法以实现规定的语义。

静态绑定：如果在运行前由编译完成这个指派，称为静态绑定

动态绑定：如果在运行期间完成这个指派，称为动态绑定

		静态绑定	动态绑定	备注
函数重载	函数重载	√		
	操作符重载	√		C++等支持
方法覆盖	虚函数		√	
	接口实现		√	Java等
	鸭子类型			解释型语言
泛型	泛型	-	-	Java等
	模板	√		C++

目录

CONTENTS

01

虚函数及其意义

02

多态的概念

03

多态-函数重载

04

多态-动态类型

05

抽象类

06

虚析构

07

RTTI运行时类型识别机制

08

final 关键字 (C++11)

09

多态的实现原理



多态——函数重载

在C++的面向对象编程中，要把编程语言结合**现实世界**进行思考。

考虑现在有3种水果：苹果、香蕉、樱桃，那么可以在c++中定义3个**类**Apple、Banana、Cherry，它们有各自的成员变量和成员函数，当然也可以用类名来声明**对象**。现在请思考：

如何用一个函数“say(fruit)”来辨别某个对象的水果种类？
可以用printf来输出水果种类。



多态——函数重载

如何用一个函数 “say(fruit)” 来辨别某个对象的水果种类？

如果学过**函数重载**，那么不难想到，可以重载多个say函数来实现。

在右方代码中，调用**同一函数** “say”，但传入不同类型的参数，会得到**不同的结果**。

```
class Apple{ /*可以随意填入成员*/ };  
class Banana{ /*可以随意填入成员*/ };  
class Cherry{ /*可以随意填入成员*/ };  
  
void say(Apple fruit){printf("I'm an apple!\n");}  
void say(Banana fruit){printf("I'm a banana!\n");}  
void say(Cherry fruit){printf("I'm a cherry!\n");}  
  
int main(){  
    Apple a;  
    Banana b;  
    Cherry c;  
    say(a);  
    say(b);  
    say(c);  
}
```




多态——函数重载

```
int abs(int x) {    // 整数类型数据的绝对值函数
    cout << "Using integer version of abs().\n";
    return (x >= 0 ? x : -x);
}

double abs(double x) { // 浮点类型数据的绝对值函数
    cout << "Using floating-point version of abs().\n";
    return (x >= 0.0 ? x : -x);
}

int main() {
    cout << abs(-5) << "\n";    // 调用abs()的整数版本
    cout << abs(3.14) << "\n";  // 调用abs()的浮点版本
    return 0;
}
```



多态——函数重载

从函数使用角度: `say(something...)`

- 由实参的数据, 函数 `say` 选择不同的行为

从函数设计角度: `say(type1,type2,...)`

- 由型参的数量和类型, 赋予函数 `say` 的不同定义
- 方便用户在不同情景下完成**相同**或**不同**的业务。例如:
 - `saveUser (int id,string name,...)`
 - `saveUser (User user)`

从多态实现角度:

- 参见 `cppreference`: 重载决议

目录

CONTENTS

01

虚函数及其意义

02

多态的概念

03

多态-函数重载

04

多态-动态类型

05

抽象类

06

虚析构

07

RTTI运行时类型识别机制

08

final 关键字 (C++11)

09

多态的实现原理



静态类型 与 动态类型

静态类型

对程序进行**编译时**分析所得到的表达式的类型被称为表达式的静态类型。程序执行时**静态类型不会更改**。

动态类型

若某个**泛左值表达式**指代某个**多态对象**，则其最终派生对象的类型被称为其动态类型。

```
struct B { virtual ~B() {} }; // 多态类型：至少包含一个虚函数
struct D: B {}; // 多态类型
D d; // 最终派生对象
B* ptr = &d;
// (*ptr) 的静态类型为 B。什么条件下 *Ptr 是静态类型？
// (*ptr) 的动态类型为 D
```



静态类型——静态绑定

从继承的特性可知，Apple、Banana、Cherry 这3个类中即使没有实现 “say” 函数，会自动继承 Fruit类的函数。

在 main 函数中，对`a.say()` 编译会延Apple继承树向上，找到最近的say成员函数定义，并翻译为`call Fruit::say(a)`，即静态联编。

所以main函数中三次调用都会输出 “I ‘m a fruit!” 。

显然，我们想要它们输出自己的水果种类，那就期望**重写** (*Override or Overwrite*) “say” 函数。

```
class Fruit{
public:
    void say(){
        printf("I'm a fruit!\n");
    }
};

class Apple : public Fruit{
};
class Banana : public Fruit{
};
class Cherry : public Fruit{
};

int main(){
    Apple a;
    Banana b;
    Cherry c;
    a.say();
    b.say();
    c.say();
}
```



静态类型——静态绑定

重写一个相同函数名、相同参数的函数，
会**覆盖或隐藏**之前继承而来的函数。那么再
次调用Apple对象的“say”函数时，就不会
输出“I ‘m a fruit!”。

这里 say 不是虚函数。所以编译器翻译
`a.say()`时 **延Apple继承树向上，找到最近**
的say成员函数定义，并翻译为`call
Apple::say(a)`，得到期望输出 “I ‘m an
apple!”

```
class Fruit{
public:
    void say() {
        printf("I'm a fruit!\n");
    }
};

class Apple : public Fruit{
public:
    void say() {
        printf("I'm an apple!\n");
    }
};

class Banana : public Fruit{
public:
    void say() {
        printf("I'm a banana!\n");
    }
};
```



静态类型——静态绑定

基类指针指向派生类对象，这对于实现多态性的行为是至关重要的。

派生类可以**重写**从基类继承过来的函数，那么请问右图的代码会输出什么？ 2选1：

- I' m a fruit!
- I' m an apple!

```
class Fruit{
public:
    void say() {
        printf("I'm a fruit!\n");
    }
};

class Apple : public Fruit {
public:
    void say() {
        printf("I'm an apple!\n");
    }
};

int main(){
    Apple a;
    Fruit *fPtr = &a;
    fPtr->say();
}
```



静态类型——静态绑定

答案是：

· I' m a fruit!

C++默认这样的成员函数“重写”为隐藏。***fPtr 是动态类型需要满足**

- 多态类型指针或引用
- 访问的成员是虚函数

既然是静态类型，编译器直接解释为

Fruit::Say(fpPtr)

```
class Fruit{
public:
    void say() {
        printf("I'm a fruit!\n");
    }
};

class Apple : public Fruit {
public:
    void say() {
        printf("I'm an apple!\n");
    }
};

int main(){
    Apple a;
    Fruit *fPtr = &a;
    fPtr->say();
}
```




动态类型——动态绑定

仅需要在**基类**的成员函数前面加上**virtual**关键字，就能把一个函数声明为**虚函数**。该类及其子类都是**多态类型**。（注意：apple也是多态类型）

```
class Fruit{
public:
    virtual void say() { printf("I'm a fruit!\n"); }
};

class Apple : public Fruit {
public:
    void say() { printf("I'm an apple!\n"); }
};

int main(){
    Apple a;
    Fruit *fPtr = &a;
    fPtr->say();
}
```

当**多态类型**指针（或引用）调用**虚函数**时，则产生多态现象，即**调用指针所指向的对象的成员函数**，上述代码输出 “I’ m an apple!”



动态类型——动态绑定

在面向对象编程中，多态与继承层次结构相关，使开发人员可以“用通用的方法编程”而不是局限于“用特殊的方法编程”。

在现实中会经常用到多态，比如“移动move”这一行为在大象和猴子中有不同表现形式，“发薪水”这一行为在教授、副教授、讲师之中可能有不同的计算方式，“绘制draw”这一行为对于三角形、正方形也是不同的。

特别是我们设计程序库时，根本**不可能知道用户行为的具体实现方式**。
多态为适应这种变化和扩展提供了便利



案例研究：Time类

要点：

识别 动态类型 和 静态类型

Time1.cpp 和 Time2.cpp 中，

- (1) 哪个类是多态类型**
- (2) 哪个变量是动态类型**
- (3) 两个程序差别在哪里**